

*Ternary* simply means an operator with three inputs, but in programming it's synonymous with *conditional expression*. Conditional expressions also offer a more concise one-liner for code that fits this pattern. In Python, they're implemented with an odd arrangement of the `if` and `else` keywords:

---

```
>>> valueIfTrue = 'Access granted'
>>> valueIfFalse = 'Access denied'
>>> condition = True
❶ >>> message = valueIfTrue if condition else valueIfFalse
>>> message
'Access granted'
❷ >>> print(valueIfTrue if condition else valueIfFalse)
'Access granted'
>>> condition = False
>>> message = valueIfTrue if condition else valueIfFalse
>>> message
'Access denied'
```

---

The expression `valueIfTrue if condition else valueIfFalse` ❶ evaluates to `valueIfTrue` if the `condition` variable is `True`. When the `condition` variable is `False`, the expression evaluates to `valueIfFalse`. Guido van Rossum jokingly described his syntax design as “intentionally ugly.” Most languages with a ternary operator list the condition first, followed by the true value and then the false value. You can use a conditional expression anywhere you can use an expression or value, including as the argument to a function call ❷.

Why would Python introduce this syntax in Python 2.5 even though it breaks the first guideline that beautiful is better than ugly? Unfortunately, despite being somewhat unreadable, many programmers use ternary operators and wanted Python to support this syntax. It's possible to abuse Boolean operator short-circuiting to create a *sort of* ternary operator. The expression `condition and valueIfTrue or valueIfFalse` will evaluate to `valueIfTrue` if `condition` is `True`, and `valueIfFalse` if `condition` is `False` (except in one important case). Enter the following into the interactive shell:

---

```
>>> # Unpythonic Example
>>> valueIfTrue = 'Access granted'
>>> valueIfFalse = 'Access denied'
>>> condition = True
>>> condition and valueIfTrue or valueIfFalse
'Access granted'
```

---